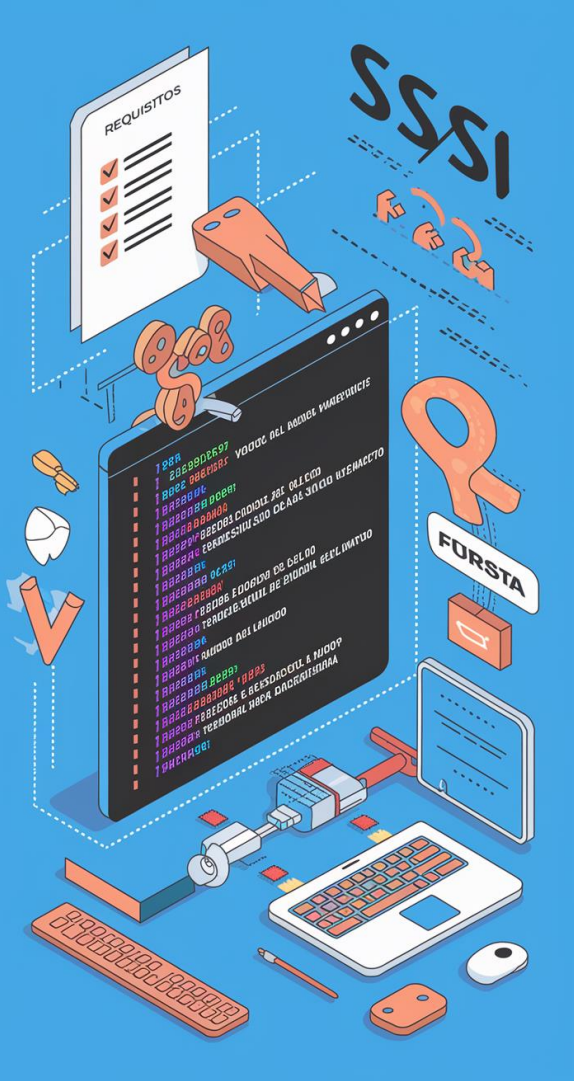


Aseguramiento de la Calidad del Software

Semana 09

Ingeniería de Software 2

2026-1





Logro de la sesión



El estudiante será capaz de comprender los conceptos de Validación y Verificación (V&V), calidad del software y tipos de pruebas, así como definir y probar requisitos mediante métodos formales.

- ¿Cómo crees que una documentación clara y bien estructurada puede influir en el éxito de un proyecto de software y en la colaboración entre los miembros del equipo?
- ¿Qué desafíos has enfrentado en proyectos anteriores debido a la falta de documentación adecuada?
- ¿De qué manera piensas que la comprensión de diferentes vistas arquitectónicas puede ayudarte a tomar decisiones más informadas y efectivas en el diseño de sistemas complejos?



Contenidos

- Validación y verificación (V&V).
- Calidad del Software.
- Métodos formales en el Software.
- Concepto básicas de prueba de software
- Tipos de pruebas de Software.

Verification and Validation (V&V)

- Procesos que aseguran que el software desarrollado **satisface las especificaciones y funcionalidades (hechas para su propósito)** solicitadas por el usuario.
- El nivel de confianza depende de:
 - Las **funciones del software**
 - Las **expectativas del usuario**
 - Y el **entorno del mercado**



Verification and Validation (V&V)

- **Verificación**

- Comprobar que el software está de acuerdo con su especificación (RF, RNF).
- **¿Estamos construyendo el producto correctamente?**
 - Salidas esperadas
 - Flujos de información correctas
 - Existencia de Bugs
 - Es más formal

- **Validación**

- Asegurar que el Software satisface las expectativas del cliente. Va más allá de la comprobación de las especificaciones del Software
- **¿Estamos construyendo el producto correcto?**
 - Corresponden las especificaciones
 - Cumplir cada requerimiento
 - Expectativas del cliente



Verification and Validation (V&V)

- **Inspecciones de Software**

- Analizan y comprueban los artefactos del software. Puede usarse las inspecciones en todas las etapas del proceso.
 - Complementadas con análisis automático de código fuente o documentos asociados
 - Las inspecciones y su complemento de análisis automáticos son **técnicas estáticas de V&V** que no necesitan ejecutar el Software

- **Pruebas de Software**

- Implica ejecutar una implementación del software con datos de prueba
- Las pruebas son una **técnica dinámica de V&V**

Inspecciones y pruebas

Las inspecciones de software implican que personas examinen la representación fuente con el objetivo de descubrir anomalías y defectos. Las inspecciones no requieren la ejecución de un sistema por lo que pueden usarse antes de la implementación. Se pueden aplicar a cualquier representación del sistema (requisitos, diseño, datos de configuración, datos de prueba, etc.). Se ha demostrado que son una técnica eficaz para descubrir errores de programas.

Las ventajas de las inspecciones incluyen:

- Durante las pruebas, los errores pueden enmascarar (ocultar) otros errores. Debido a que la inspección es un proceso estático, no es necesario preocuparse por **las interacciones entre errores**.
- **Las versiones incompletas** de un sistema se pueden inspeccionar sin costes adicionales. Si un programa está incompleto, entonces es necesario desarrollar arneses de prueba especializados para probar las piezas disponibles.
- Además de buscar defectos en el programa, una inspección también puede considerar **atributos de calidad más amplios** de un programa, como el cumplimiento de estándares, la portabilidad y la mantenibilidad.

Inspecciones y pruebas

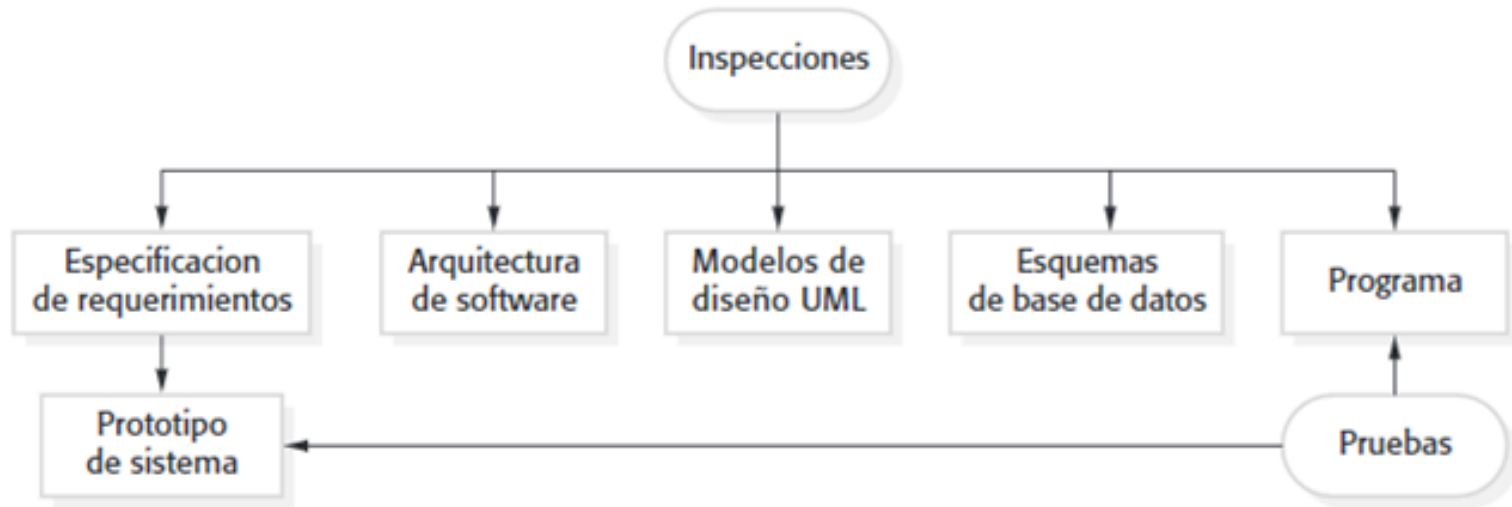
Las inspecciones y pruebas son técnicas de verificación complementarias y no opuestas. Ambos deben usarse durante el proceso V & V. Las inspecciones pueden comprobar la conformidad con una especificación, pero no la conformidad con los requisitos reales del cliente. Las inspecciones no pueden comprobar características no funcionales como rendimiento, usabilidad, etc.

Normalmente, un sistema de software comercial debe pasar por **tres etapas de prueba** :

- **Pruebas de desarrollo** : el sistema se prueba durante el desarrollo para descubrir errores y defectos.
- **Pruebas de lanzamiento** : un equipo de pruebas separado prueba una versión completa del sistema antes de lanzarlo a los usuarios.
- **Pruebas de usuario** : los usuarios o usuarios potenciales de un sistema prueban el sistema en su propio entorno.

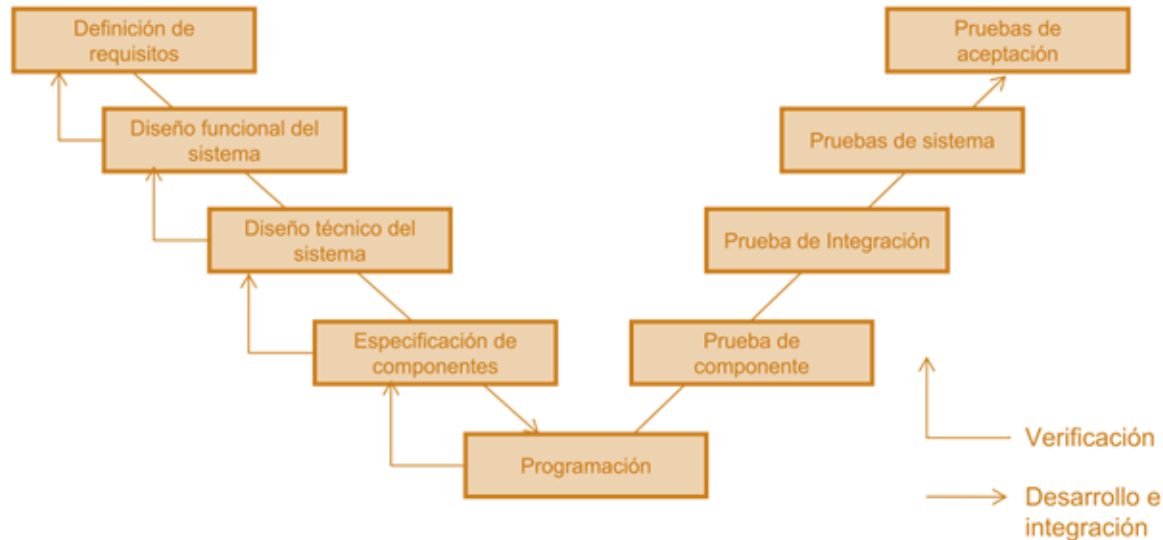
Verification and Validation (V&V)

¿Cuándo hacer inspecciones y pruebas?



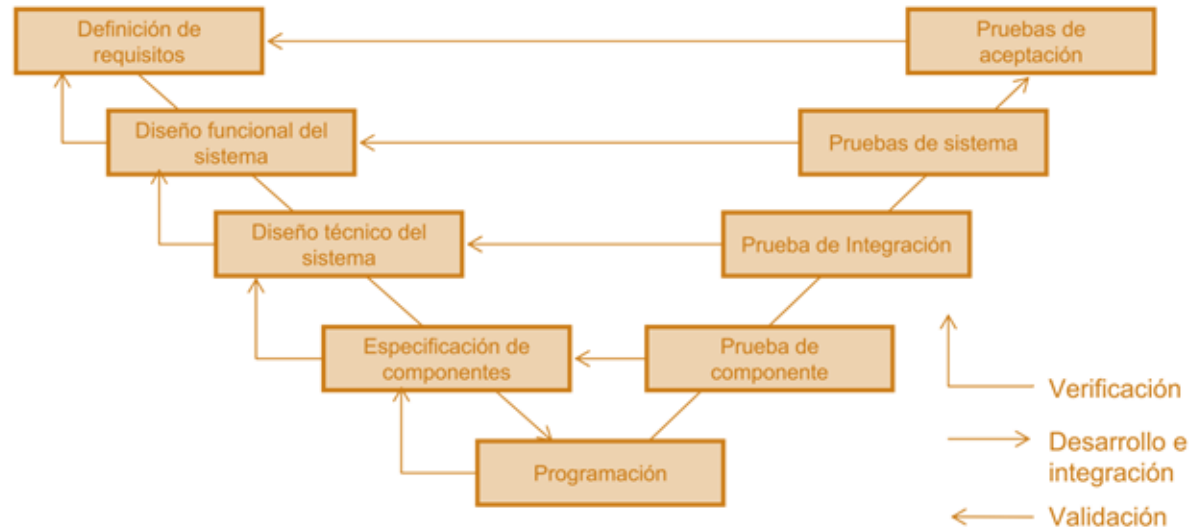
Verificación dentro del modelo-V general

- Cada nivel de desarrollo se verifica respecto de los contenidos del nivel que le precede.
- Verificar significa comprobar si los requisitos y definiciones de niveles previos han sido implementados correctamente.



Validación dentro del modelo-V general

- La validación se refiere a la correctitud de cada nivel de desarrollo.
- Validar significa comprobar lo adecuado de los resultados de un nivel de desarrollo.



Calidad del Software

Es un concepto complejo y no se puede comparar directamente con la calidad de productos manufacturados. Los productos de software son estratégicos para muchas organizaciones, ya que los procesos clave dependen de su correcto funcionamiento.

Algunas Definiciones de Calidad

- Propiedad o conjunto de propiedades inherentes a un objeto que permiten apreciarlo como mejor, igual o peor que otros objetos de su especie [DRAE: Diccionario de la Real Académica Española]
- Conjunto de propiedades y de características de un producto o servicio que le confieren capacidad para satisfacer necesidades expresadas o implícitas. [ISO 8042:1994]
- Grado en el que un conjunto de características inherentes cumple con los requisitos. [ISO 9000: 2000]
-

Algunas Definiciones de Calidad

- Calidad, significa desarrollar, diseñar y producir y mantener un producto que sea el más económico, el más útil y siempre satisfactorio para el consumidor. [Kaoru Ishikawa 1943, ¿Qué es el control total de calidad?: la modalidad japonesa.]
- Calidad, es la aplicación de los principios y técnicas estadísticas en todas las fases de la producción, dirigida a la fabricación más económica de un producto (o servicio) que es útil en grado máximo y que tiene mercado. [William Edwards Deming 1986, Out of the Crisis. MIT Press]

Calidad

- No es absoluto
- Está sujeto a restricciones
- Trata de compromisos aceptables
- Es multidimensional
- Los criterios de calidad no son independientes

Introducción a la Calidad de Software

La calidad del software se refiere al **grado** en que un sistema o componente **cumple** con los **requisitos** y expectativas del usuario.

Para lograr un software de calidad, es esencial utilizar metodologías estándar en análisis, diseño, programación y pruebas, lo que mejora la confiabilidad, mantenibilidad y productividad en el desarrollo y control de calidad. A diferencia de la manufactura, la calidad del software depende más del proceso de diseño y de las habilidades del individuo. En productos innovadores, la capacidad del equipo es más crucial que el proceso en sí.



Introducción a la Calidad de Software

- Definición: "La calidad del software es el grado en que un sistema cumple con los requisitos y expectativas del usuario" (IEEE Standard Glossary of Software Engineering Terminology)
- Importancia: Los sistemas de software son estratégicos para las organizaciones, ya que los procesos clave dependen de su correcto funcionamiento. .).



Aspectos de la Calidad de Software

Basado en el modelo de calidad descrito en la norma ISO/IEC 9126

- **Calidad interna:** Se mide a partir de características intrínsecas, como el código fuente.
- **Calidad externa:** Se evalúa en el comportamiento del producto, como en pruebas.
- **Calidad en uso:** Se mide durante la experiencia real del usuario.



Metodologías y Estándares

Uso de metodologías estándar para análisis, diseño, programación y pruebas.

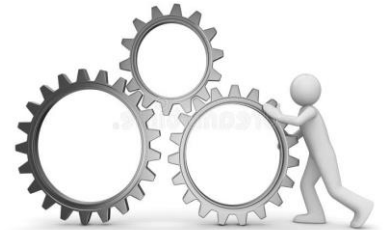
Normas clave:

- **ISO/IEC 9126:** Define las características de calidad del software (funcionalidad, confiabilidad, usabilidad, eficiencia, mantenibilidad y portabilidad).
- **ISO/IEC 14598:** Proporciona directrices para la evaluación de la calidad del software.



Factores Clave en la Calidad de Software

- **Confiabilidad:** El software debe funcionar correctamente bajo condiciones específicas.
- **Mantenibilidad:** Facilidad para realizar modificaciones o correcciones.
- **Facilidad de prueba:** Capacidad de verificar el correcto funcionamiento del software.
- **Productividad:** Mejora en el desarrollo y control de calidad.



El Rol del Proceso y del Equipo

- A diferencia de los productos físicos, la calidad del software no depende de un proceso de manufactura repetitivo, sino de un proceso de diseño creativo y técnico.
- En la manufactura, la calidad está determinada principalmente por la maquinaria y los procesos estandarizados. En el software, el diseño y las decisiones tomadas durante el desarrollo son los factores clave.
- En proyectos innovadores, las habilidades, experiencia y creatividad del equipo son más importantes que el proceso en sí.
- Un equipo capacitado y colaborativo puede adaptarse a desafíos y encontrar soluciones efectivas, incluso en entornos con procesos menos definidos.



¿Cómo logramos esto?

- **Aseguramiento de la calidad del SW**
 - Gestión de la configuración del SW
 - Planificación y control
 - Evaluación y control de riesgos
 - Revisiones del sw
 - Pruebas del sw.
- **Uso de Métodos Formales**



Métodos formales

Utilizados para especificar, desarrollar y verificar el software mediante la aplicación de una notación y lenguaje matemático riguroso.

Métodos Formales

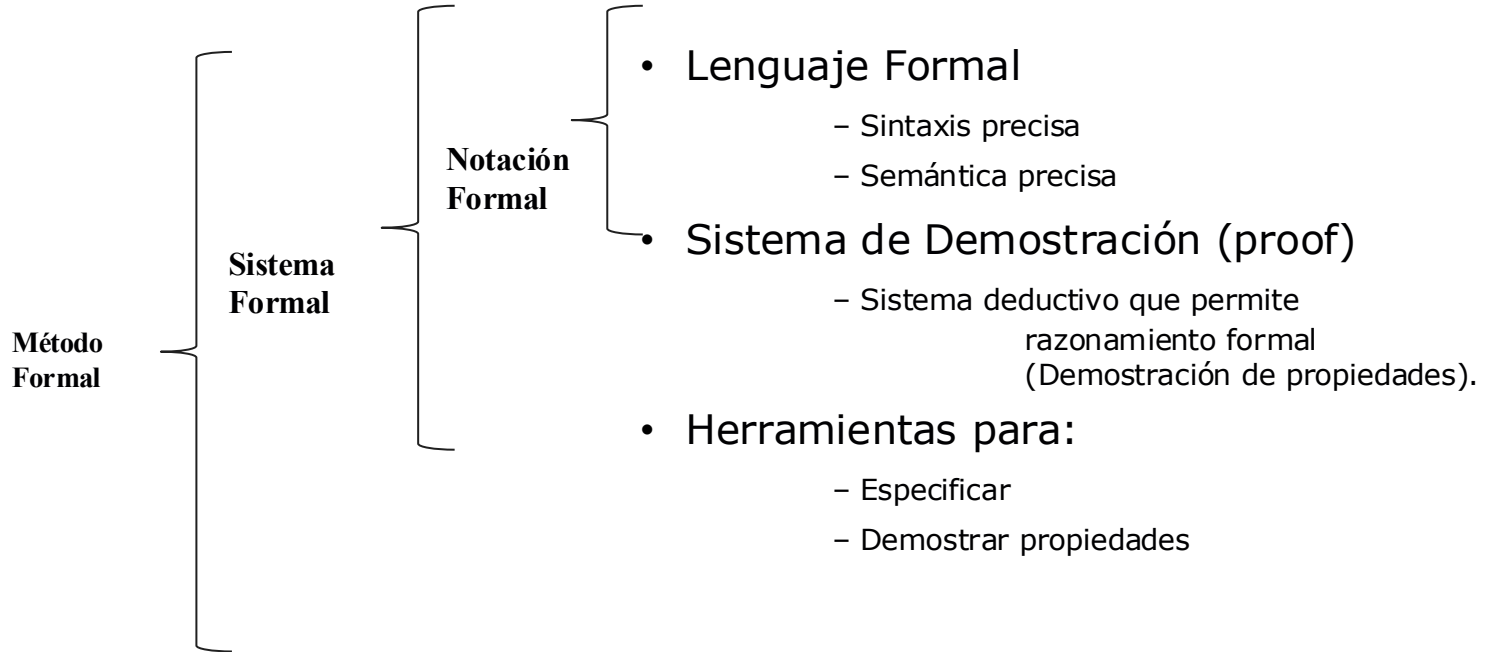
“Los métodos formales que se utilizan para desarrollar sistemas de computadoras, son técnicas de base matemática para describir propiedades del sistema. Estos métodos formales **proporcionan marcos de referencia** en el seno de los cuales las personas pueden especificar, desarrollar y verificar los sistemas de manera sistemática, en lugar de hacerlo ad-hoc”.

Software Engineering Encyclopedia (Marcianiack, JJ; 1994).

Métodos Formales

"Técnicas basadas en matemáticas para la especificación, el desarrollo y verificación de software y sistemas de hardware".

MF, Sistema Formal y Notación Formal



Tres niveles de los métodos formales

LEVEL 0
Formal Specification



1. Solo requisitos
2. Sin análisis / prueba
3. Rentable

LEVEL 1
Formal Verification



1. **Producir un programa** de una manera más formal
2. **Usa pruebas** de propiedades o refinamientos de la especificación formal
3. Costoso

LEVEL 2
Theorem Provers



1. Usa el **demostrador de teoremas**
2. **Máquina completamente formal** comprobaciones comprobadas.
3. Caro, difícil y a menudo costoso
4. Demuestra formalmente todo el sistema.



Uso en el proceso de Diseño

- **Especificación**
- **Desarrollo**
- **Verificación**

Uso en el proceso de diseño

- **Especificación**

Descripción del sistema que se desarrollará en cualquier nivel de detalle deseado..

La especificación formal se puede utilizar:

1. para **guiar el desarrollo** posterior.
2. **verificar los requisitos** del sistema se especifican completamente y con precisión.

Uso en el proceso de diseño

- **Desarrollo**

Las especificaciones formales se pueden usar como una guía para progresar en el desarrollo del sistema real.

Especificación orientada a modelo

- Comportamiento observado del sistema real en comparación con el comportamiento de la especificación.

Especificación orientada a la propiedad

- Las precondiciones y poscondiciones pueden convertirse en aserciones en el código de sistema ejecutable.

Uso en el proceso de diseño

- **Verificación**

La especificación formal se puede usar para **probar ciertas propiedades de especificación** y, por lo tanto, el sistema desarrollado..

Prueba dirigida - Humano

- **Comprende mejor** el sistema
- **Pruebas de estilo** matemático
- **Manuscrito** usando un lenguaje natural

Prueba automatizada

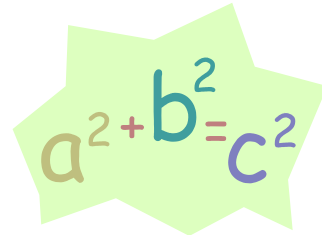
- **Producir pruebas usando** medios automatizados
- ***Prueba de teorema automatizado***
- ***Verificación de modelo***

Lenguaje Formal

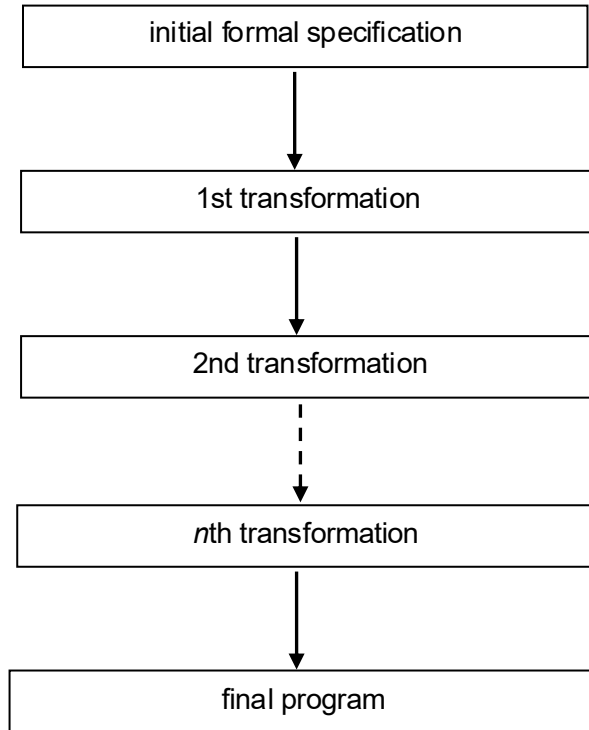
Es deseable usar una notación de especificación con una **semántica fija, no ambigua.**

Las **anotaciones** que tienen una **semántica** fija se conocen como **notaciones formales o lenguajes formales.**

Se **logra una semántica fija** al definir un idioma de una manera completamente inequívoca utilizando **un marco matemático.**


$$a^2 + b^2 = c^2$$

Método Formal



Un método formal incluye un **sistema de prueba** para demostrar que cada transformación conserva el significado formal capturado en el paso anterior.

Ventajas de los Métodos Formales

- las especificaciones formales pueden ayudar considerablemente a **generar casos de prueba** adecuados;
- la disciplina requerida para producir una especificación formal permite la **retroalimentación** sobre las especificaciones del sistema en las primeras etapas de desarrollo, aumentando la confianza de que la especificación captura con precisión los requisitos reales del sistema;
- las propiedades importantes de la especificación inicial **pueden verificarse matemáticamente** e incorporarse como verificaciones en tiempo de ejecución en el programa final;
- las pruebas pueden ayudar a **descubrir errores de diseño** tan pronto como se realicen, en lugar de tener que esperar para probar la implementación final;
- Se puede **construir una prueba de corrección** del programa que es un **método mucho más robusto** para lograr la corrección del programa que solo la prueba

Ventajas de los Métodos Formales

- Los métodos formales **tratan los componentes del sistema como objetos matemáticos** y proporcionan modelos matemáticos para **describir** y **predecir** las propiedades observables y los comportamientos de estos objetos.
- El uso de métodos formales para la especificación y el análisis de sistemas en tiempo real presenta varias ventajas.
 - el **descubrimiento temprano de ambigüedades**, inconsistencias e incompletitud en los requisitos informales
 - el **análisis automático o asistido por máquina** de la corrección de las especificaciones con respecto a los requisitos
 - la **evaluación de alternativas de diseño** sin costosos prototipo

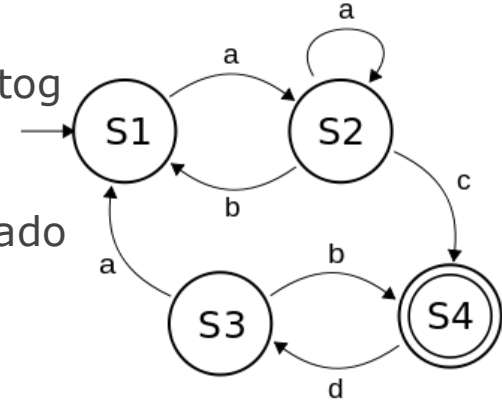
Métodos de especificación Formal

- **Lógica**

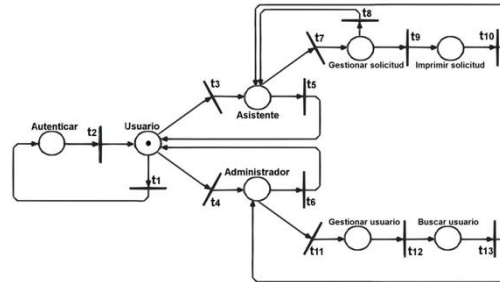
- Z, VDM, First order logic, temporal logic, setog
{o}

- **Máquina de estados**

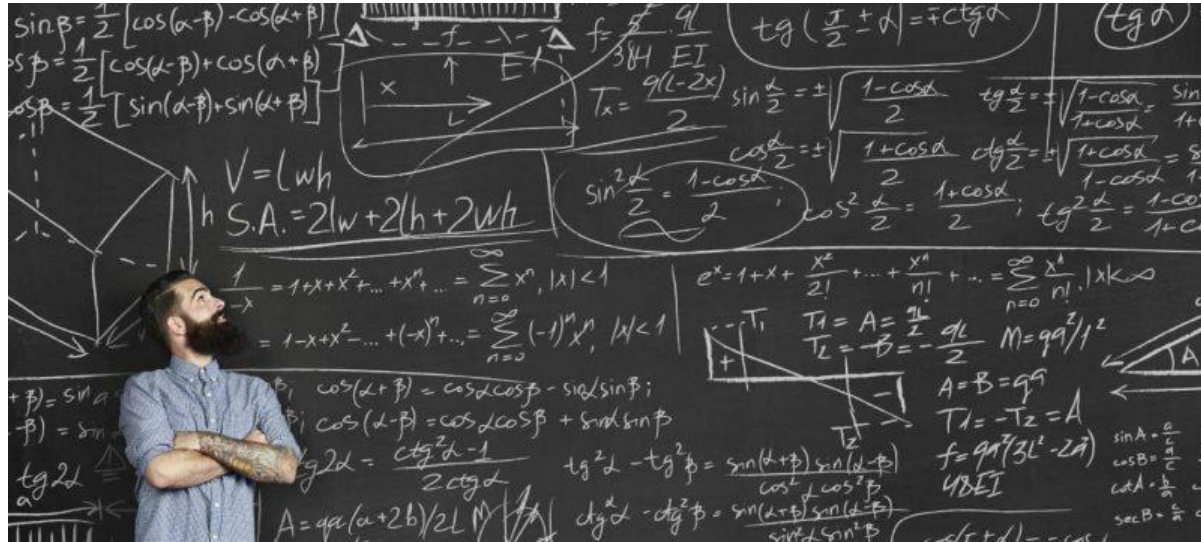
- Máquinas de estado finito, máquinas de estado de comunicación, máquinas de estado extendido
- Gráfico de estado, Objectime, Autómatas



- **Redes de Petri**

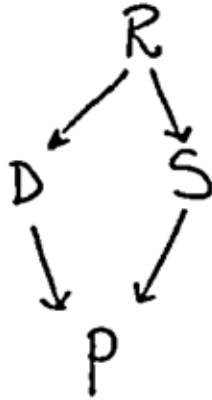


Un poco de matemáticas...



Matemática ...

¿MATEMÁTICA PARA PROGRAMAR?



R → REQUISITOS, DOCUMENTO EN CASTELLANO

D → DISEÑO, DIVISIÓN DEL CÓDIGO (CLASES, MÓDULOS,
PATRONES DE DISEÑO)

P → PROGRAMA, CÓDIGO FUENTE

S → ESPECIFICACIÓN MATEMÁTICA

ESPECIFICACIÓN FORMAL

CULEAR: MÉTODOS FORMALES

FORMAL METHODS

Matemática ...

TEORÍA DE CONJUNTOS (FINITOS)

CONJUNTO : CONTENEDOR DE DATOS
SIN ORDEN
SIN REPETICIONES

CONJUNTO VACÍO $\rightarrow \emptyset$ o $\{\}$

CONJUNTO POR EXTENSIÓN $\{1, 4, 2\}$ $\{2, 6\}$

SIN ORDEN : $\{1, 4, 2\} = \{4, 1, 2\} = \{2, 1, 4\}$

SIN REPETICIONES : $\{1, 4, 2, 4\} = \{1, 4, 1, 2, 4\} = \{1, 4, 2\}$

Matemática ...

OPERACIONES CON CONJUNTOS

Unión

$$\{1, 3\} \cup \{2\} = \{1, 3, 2\}$$

$$\{1, 3\} \cup \{1\} = \{1, 3, 1\} = \{1, 3\}$$

$$\{1, 3\} \cup \emptyset = \{1, 3\}$$

$$\{1, 3\} \cup \{11, 5\} = \{1, 3, 11, 5\}$$

Funciones

Toda función es un conjunto de pares ordenados

NO todo conjunto de pares ordenados es una función

$$\{[1, 7], [4, 15], [1, 21]\}$$

NO es función porque hay dos pares ordenados distintos que tienen la misma primera componente

Las funciones son muy importantes para especificar sistemas

↓
Como conjuntos de pares ordenados

Ejemplo...

instalar intérprete de Prolog

pueden instalar SWI-Prolog que es gratuito

<http://www.swi-prolog.org/>

luego instalar {log}, descargarlo aquí

<http://people.dmi.unipr.it/gianfranco.rossi/setlog.Home.html>



Ejemplo...

RESTA O DIFERENCIA DE CONJUNTOS

$A \setminus B$ es el conjunto formado por todos los elementos de A que no están en B

Es decir, sacamos de A todos los elementos que también son elementos de B .

$$A = \{1, 2, 3\} \quad B = \{2\} \rightarrow A \setminus B = \{1, 3\}$$

$$B = \{1, 3\} \rightarrow A \setminus B = \{2\}$$

$$B = \{4\} \rightarrow A \setminus B = \{1, 2, 3\}$$

$$B = \{3, 4\} \rightarrow A \setminus B = \{1, 2\}$$

Ejemplo...

Aplicación DE FUNCIÓN

F ES UNA FUNCIÓN $\rightarrow$ CONJUNTO DE PARES ORDENADOS

$$F = \{ \dots, [x, y], \dots \}$$

Entonces $F(x)$ ES IGUAL A LA SEGUNDA COMPONENTE
DEL PAR ORDENADO DE F CUYA PRIMERA COMPONENTE ES x .

En ESTE CASO $F(x) = y$

$$F = \{ [5, 25], [7, 49], [2, 8], [10, 101] \}$$

$$F(7) = 49$$

$$F(2) = 8$$

$$F(3) \text{ INDETERMINADO}$$

$3 \notin \text{DOM } F$

Ejemplo...

Restricción de dominio

$\mathcal{R} \rightarrow$ Conjunto de pares ordenados $\rightarrow [z, y]$

don $\mathcal{R} \rightarrow$ dominio de $\mathcal{R} \rightarrow$ conjunto de las primeras
componentes de los pares de $\mathcal{R}$

$A \triangleleft \mathcal{R} \rightarrow$ restricción de dominio de $\mathcal{R}$ al conjunto A

$A \triangleleft \mathcal{R} \rightarrow$ los pares ordenados de $\mathcal{R}$ cuya primera
componente pertenece a A

$$A \triangleleft \mathcal{R} = \{ [z, y] \in \mathcal{R} \mid z \in A \}$$

Ejemplo...

Restricción de dominio

$$R = \{[3, 7], [4, 2], [3, 9]\}$$

$$A = \emptyset \rightarrow A \triangleleft R = \emptyset$$

$$A = \{9\} \rightarrow A \triangleleft R = \emptyset \quad 9 \notin \text{dom } R$$

$$A = \{3\} \rightarrow A \triangleleft R = \{[3, 7], [3, 9]\}$$

$$A = \{4, 12\} \rightarrow A \triangleleft R = \{[4, 2]\} \quad 12 \notin \text{dom } R$$

$$A = \{4, 3\} \rightarrow A \triangleleft R = R = \{[3, 7], [4, 2], [3, 9]\}$$

$A = \text{dom } R$

Ejemplo...

Anti-RESTRICCIÓN DE DOMINIO

$A \triangleleft R \rightarrow$ Anti-RESTRICCIÓN DE DOMINIO DE R
AL conjunto A

$A \triangleleft R \rightarrow$ CONJUNTO DE PARES DE R CUYA PRIMERA
COMPONENTE NO PERTENECE A A

$$A \triangleleft R = \{[x, y] \in R \mid x \notin A\}$$

PARA CUALESQUIERA R Y A VALE QUE

$$R = (A \triangleleft R) \cup (A \triangleleft R)$$

Ejemplo...

Anti-RESTRICCIÓN DE DOMINIO

$$R = \{ [3,7], [4,2], [3,9] \}$$

$$A = \emptyset \rightarrow A \triangleleft R = R = \{ [3,7], [4,2], [3,9] \} \quad A \triangleleft R = \emptyset$$

$$A = \{9\} \rightarrow A \triangleleft R = R \quad A \triangleleft R = \emptyset$$

$$A = \{3\} \rightarrow A \triangleleft R = \{ [4,2] \} \quad A \triangleleft R = \{ [3,7], [3,9] \}$$

$$A = \{4,12\} \rightarrow A \triangleleft R = \{ [3,7], [3,9] \} \quad A \triangleleft R = \{ [4,2] \}$$

$$A = \{4,3\} \rightarrow A \triangleleft R = \emptyset \quad A \triangleleft R = R$$

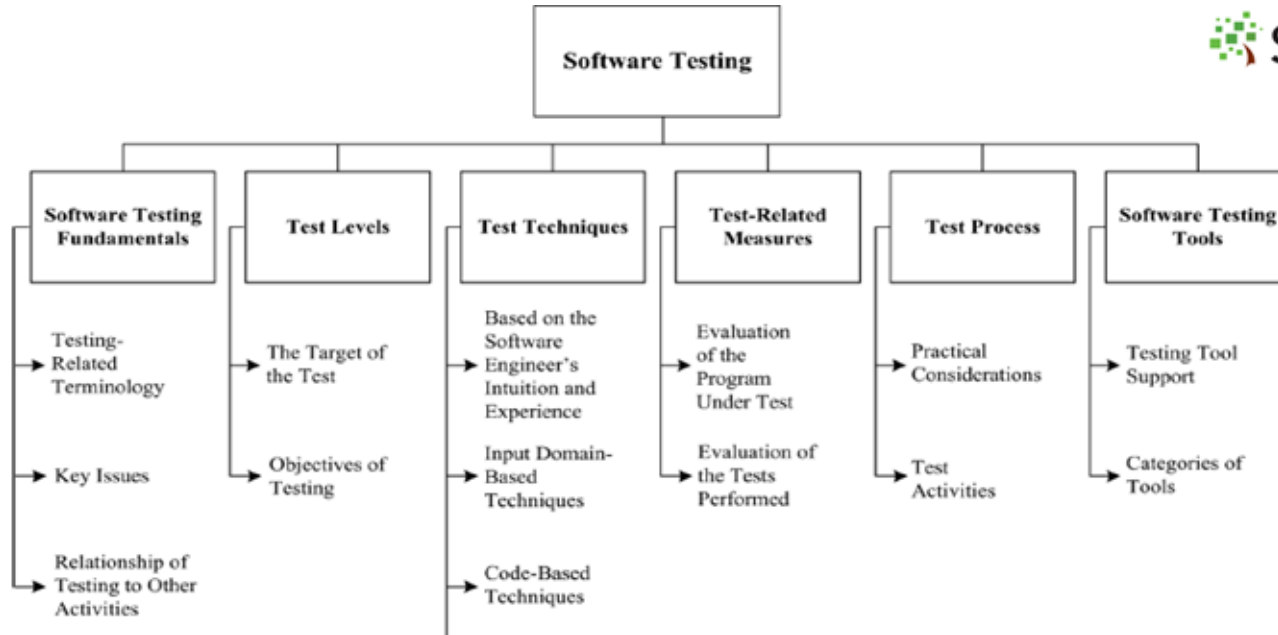
Conceptos básicos de pruebas

Pruebas de Software

- ¿Qué es una prueba de Software?
 - Proceso que busca demostrar que el sistema hace lo que debe hacer, localizando errores y/o anomalías.



Software Testing según SWEBOK V3.0



Certifying Software Testers Worldwide

AGILE

CORE

SPECIALIST

EXPERT LEVEL

Test Management

Managing The Test Team

Operational Test Management

Strategic Test Management

Improving The Test Process

Assessing Test Processes

Implementing Test Process Improvement

ADVANCED LEVEL

Technical Test Analyst

Test Analyst

Test Manager

FOUNDATION LEVEL

Certified Tester

ADVANCED LEVEL

Agile Technical Tester

Agile Test Leadership At Scale **MVP**

FOUNDATION LEVEL

Agile Tester

Acceptance Testing

AI Testing

Automotive Software Tester

Gambling Industry Tester

Game Testing **BETA**

Mobile Application Testing

Model-Based Tester

Performance Testing

Security Tester

Test Automation Engineer

Usability Testing



<https://www.istqb.org/>

START HERE



**“Las pruebas sólo
pueden mostrar la
presencia de
errores,
mas no su
ausencia.”**



(Dijkstra et al., 1972)

Conceptos Generales

- **Defecto (defect)**
 - Es el resultado de una deficiencia o error durante la construcción del software.
 - Elemento fuera de rangos normales
 - Pertenece a algo más grande
 - No corresponde a sus especificaciones
- **Fallo (failure)**
 - Es la incapacidad del software (o una de sus partes) de realizar sus funciones específicas.
 - Se ha ocasionado un resultado no esperado
 - Hay (¡o no hay!) un mensaje de error
 - Una interrupción

610.12-1990 - IEEE Standard Glossary of Software Engineering Terminology



Conceptos Generales

- **Error (bug, fault)**
 - Una equivocación cometida por el desarrollador.
 - Pueden ser:
 - De requisitos
 - De diseño
 - De implementación



Relación entre Error, Defecto y Fallo



Errores comunes

- División entre cero
- Ciclo infinito
- Problemas aritméticos como desbordamientos (overflow)
- Condición de carrera
- Exceder el tamaño del arreglo
- Utilizar una variable no inicializada
- Acceder a memoria no permitida (Violación de acceso)
- Pérdida de memoria (memory leak)
- Desbordamiento o subdesbordamiento del arreglo
- Desbordamiento de búfer (buffer overflow)
- Bloqueo mutuo (deadlock)
- Indizado inadecuado de tablas en bases de datos

¿Quién debe hacer las pruebas?

- El equipo de desarrollo
 - Conoce cómo debe ser el funcionamiento del sistema.
 - Puede tener un sesgo al probar (evita comportamientos raros)
 - Puede ser más económico (a corto plazo)
- Un equipo sólo de pruebas
 - Personas especializadas en calidad
 - Es más costoso
 - No tienen sesgos al probar (más objetivos)
 - Paralelizable con desarrollo



Conceptos Generales :Depuración (debugging)

Proceso para corregir los errores y problemas encontrados por las pruebas.

- Eliminar los bugs encontrados.
- Usar las salidas de las pruebas (mensajes, resultados esperados, datos introducidos) para brindar soluciones.
- Realizadas por el equipo de desarrollo.
- Se debe dar un tiempo estimado (explícito) en el cronograma.



Conceptos Generales : Pase a producción

- **Pase a producción**
 - Proceso de instalación, implantación o puesta a disposición de un Software en el ambiente donde debe realizar sus funciones.
 - Culminadas las pruebas.
 - Pasar del entorno de desarrollo al entorno final.
 - Pueden haber entornos de pre-producción intermedios.
 - Dependiendo de la complejidad del sistema, puede ser un proceso largo o muy directo.



Tipos de pruebas

Tipos de Pruebas

- Podemos clasificarlas por granularidad, por el método para aplicarlas, por las técnicas que se usan, por los objetivos que se buscan, etc.

Tipos de Pruebas: Prueba de Desarrollo

Las pruebas de desarrollo abarcan todas las actividades de verificación y validación realizadas durante el proceso de creación del software. Su objetivo es garantizar que el producto cumpla con los requisitos especificados y funcione correctamente en cada etapa del desarrollo.

Clasificación por granularidad incluyen:

- Pruebas de Unidad
- Pruebas de Componente
- Pruebas de Sistema

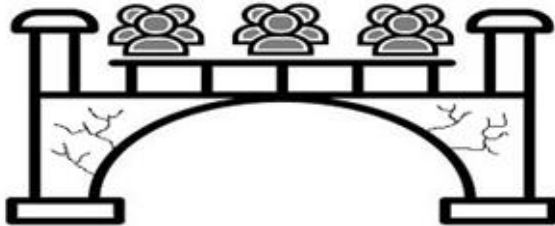
Tipos de Pruebas: Prueba de Desarrollo

Clasificación por fases del desarrollo (en el ciclo de vida):

- Pruebas unitarias
- Pruebas de integración
- Pruebas de sistema
- Pruebas de regresión

Tipos de Pruebas

- **Pruebas de componente:** Se enfocan en el uso de las interfaces entre los componentes. Además, se pone a prueba la arquitectura del software.
 - Pueden detectar:
 - Uso incorrecto de interfaz
 - Mala interpretación de la interfaz
 - Errores temporización (carrera)
 - Se pueden usar **pruebas de estrés** en esta etapa.



Prueba de Carga



Prueba de Estrés



Tipos de Pruebas

- **Pruebas de componente**
 - Lineamientos
 - Listar llamados de interfaces
 - Forzar llamados a nulos (si no está el componente)
 - Probar que pasa cuando falla el otro componente
 - Pruebas de esfuerzo al enviar mensajes
 - Pruebas de memoria compartida

Tipos de Pruebas

- **Pruebas del Sistema**
 - Se prueba el Sistema por Completo
 - Se dan las pruebas de integración completa
 - Pruebas usando los casos de uso/actividades:
 - -Colocar todas las entradas, incorrectas o correctas
 - -Seguir el proceso completo
 - -Probar todas las combinaciones

Tipos de Pruebas

- **Prueba de Integración:** Las pruebas de integración son parte de las pruebas de sistema. Para integrar, se pueden usar diferentes técnicas:
 - TopDown
 - BottomUp
 - Sandwich
 - BigBang

Tipos de Pruebas

- **Prueba de Versión:** Pruebas que se pueden realizar para cada release (lanzamiento) de un sistema. Dependen mucho del tipo de sistema y proyecto.
 - Pruebas basadas en requerimientos
 - Pruebas de escenarios
 - Pruebas de rendimiento
 - Pruebas de regresión
 - Pruebas de aseguramiento de calidad de datos

Tipos de Pruebas

- **Pruebas basadas en requerimientos :** Analizamos cada requerimiento y se diseña una prueba por cada uno. Los requerimientos deben ser medibles y específicos.
- **Pruebas de escenarios:** Generar escenarios según roles en el sistema. Especificar interacciones con el usuario.
- **Pruebas de aseguramiento de calidad de datos:** Revisar que los datos corresponden a lo esperado. Puede ser para nuevos sistemas o migraciones.

Tipos de Pruebas

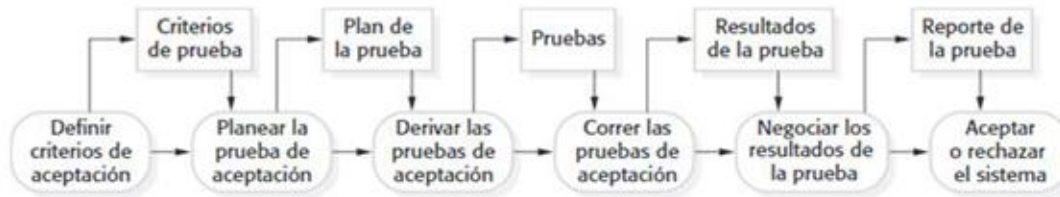
- **Pruebas rendimiento**
 - Prueba de esfuerzo (estrés)
 - Necesariamente requiere automatización. Ej: Jmeter, AB, Siege.
 - Se debe aguantar más de lo máximo necesario.
 - Prueba de seguridad
 - Se sigue un protocolo para probar capas (red) del sistema.
- **Pruebas de regresión**
 - Evaluar que el cambio de versión no rompe funcionalidad anterior.
 - Puede incluir otras pruebas.

Tipos de Pruebas

- **Pruebas de Usuario**

- Cuando el usuario (cliente) apoya en las pruebas del sistema.

- **Pruebas Alfa:** Se trabaja con un sistema que aún está en desarrollo, junto con el equipo.
- **Pruebas Beta:** Se permite el uso libre de usuarios al sistema para que experimenten.
- **Pruebas de Aceptación:** Se decide si el sistema ya pasa a producción. En XP (y otras metodologías ágiles), las pruebas de aceptación son parte del desarrollo del sistema, por lo que no existe como paso separado.



Elaboración de Pruebas de Aceptación

- **¿Qué son las pruebas de aceptación?**
 - También se les conoce como pruebas de usuario.
 - Nos permiten determinar si lo que estamos construyendo cumple con los requerimientos de nuestros usuarios.
 - Normalmente se manejan como pruebas de caja negra (punto de vista de usuario), abstrayéndose de todo detalle de implementación.
 - Normalmente al cumplir con estas pruebas, se considera que el software es aceptado por nuestros usuarios.

Elaboración de Pruebas de Aceptación

- Tipo

Funcional

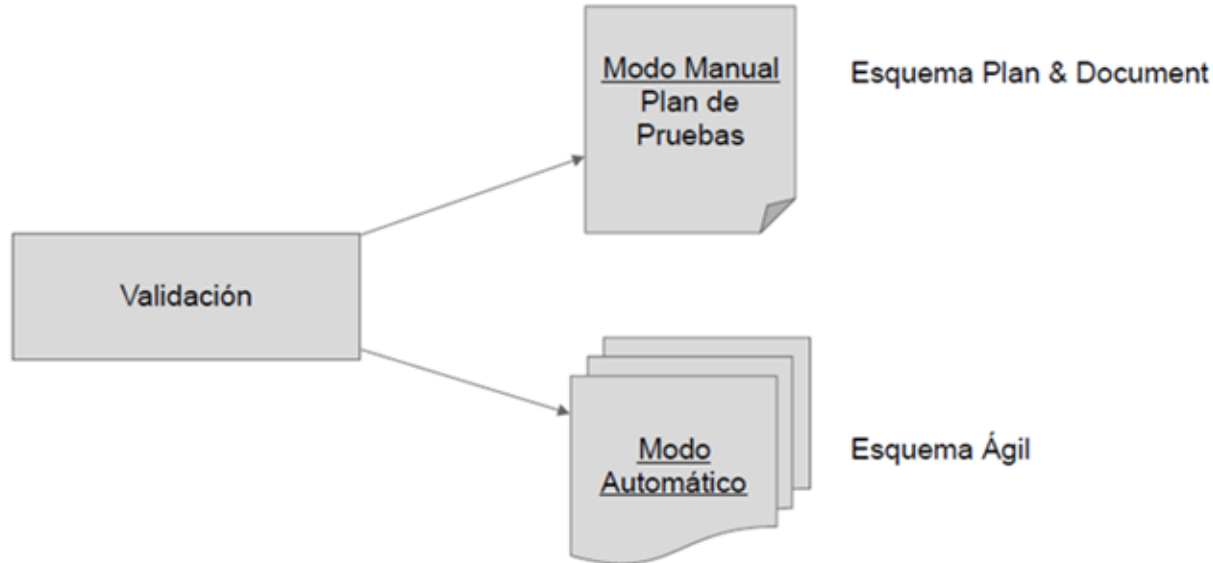


Usabilidad



Elaboración de Pruebas de Aceptación

- Estrategia para pruebas de aceptación



Elaboración de Pruebas de Aceptación

- **Plan de Pruebas Funcionales**
 - Bajo el enfoque Plan & Document, al iniciar el proyecto (de preferencia) se deberá de implementar un documento llamado Plan de Pruebas que cuente:
 - Casos de pruebas con sus escenarios (instrucciones para realizar la prueba y criterios de aceptación).
 - Escenarios no contemplados en las pruebas.
 - Riesgos
 - Participantes (tanto internos como externos).

Elaboración de Pruebas de Aceptación

- **Plan de Pruebas de Usabilidad**
 - Se trata de evaluar el comportamiento de los usuarios durante el uso del producto de software.
 - Usualmente durante los tests se pide que ciertos usuarios realizan tareas comunes con el software para que así los miembros del equipo puedan hacer notas del comportamiento de estos.
 - Objetivos:
 - Descubrir problemas de usabilidad.
 - Obtener data cuantitativa y cualitativa que nos permitan valorar la satisfacción de nuestros usuarios.

Estrategia para las pruebas según el nivel de las actividades

- Realizaremos pruebas en diferentes etapas del desarrollo de software.
- Comenzaremos a un nivel bastante alto y progresivamente iremos descendiendo en nuestros niveles de abstracción.

Pruebas de Aceptación

Pruebas de Sistema

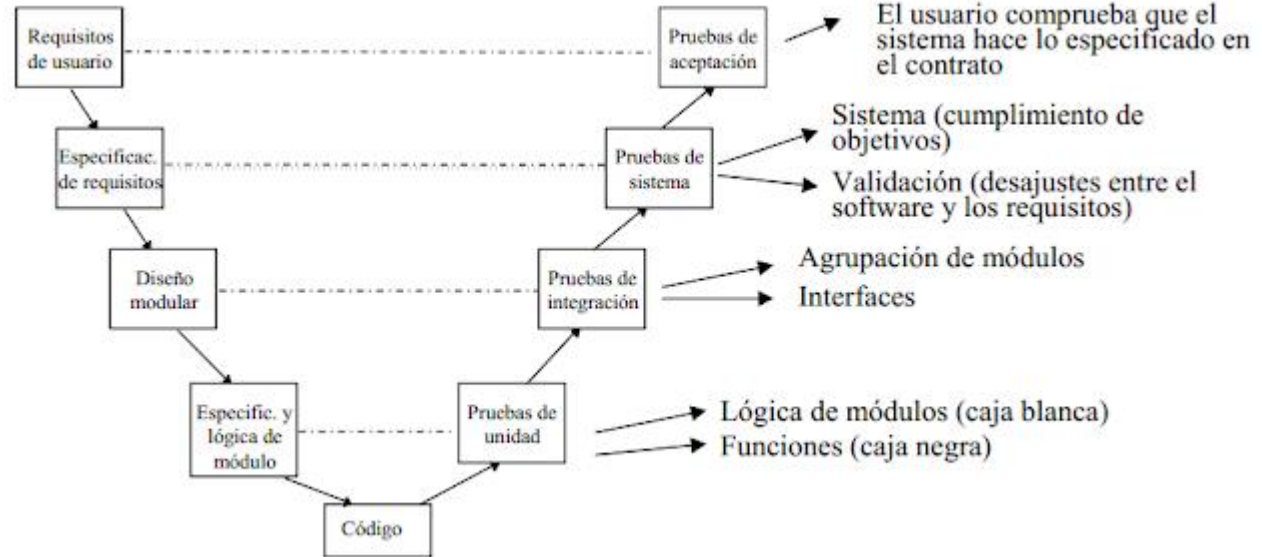
Pruebas de Integración

Pruebas Modulares

Pruebas Unitarias

Actividades de Software vs Niveles de Testing (Modelo V)

- Requerimientos (features) del producto.
- Pruebas de la comunicación entre los distintos componentes del sistema (componentes de más alto nivel).
- Pruebas de la comunicación entre los distintos módulos (clases).
- Pruebas de los módulos (clases).
- Pruebas de las funciones.



Tomado de: <https://scrum-ga.blogspot.com/2013/03/modelos-de-desarrollo-de-software.html>

¿Qué se logró?



- ✓ Comprender los conceptos fundamentales de Validación y Verificación (V&V) y su importancia en la calidad del software.
- ✓ Identificar y diferenciar los distintos tipos de pruebas de software y su aplicación en el aseguramiento de la calidad.
- ✓ Definir y probar requisitos utilizando métodos formales para verificar la corrección del software."

¿Qué debemos hacer para alcanzar el objetivo del tema?



- ✓ Repasar la teoría, los ejemplos y casos resueltos.



Referencias Bibliográficas

- Proceso de Construcción de Software 2, Mag. Natalí Flores Lafosse, Maestría en Informática de la PUCP, 2018.
- Sommerville, I. (2005). Ingeniería del software. Pearson educación.
- Bourque, P., & Fairley, R. E. (2014). Guide to the software engineering body of knowledge (SWEBOK (R)): Version 3.0. IEEE Computer Society Press.
- Doria, H. G. (2001). Las Metricas de Software y su Uso en la Region.

GRACIAS